

What is animation?

Animation is a technique that produces a stop-and-move video. It captures a snapshot of the execution state (a partial result) at time t , performs certain actions, and then takes a snapshot of the next state. The display sequences these snapshots in strict temporal order, creating an illusion of motion. However, the transitions occur more slowly than in a real video, allowing viewers to grasp the logic behind the program's execution. Essentially, the video updates the snapshot to time $t + 1$ as the program moves to the next state.

Let us consider the animation of a sorting process. We can create widgets to represent numbers. Since these numbers are comparable, the corresponding widgets must also be comparable in their geometric dimensions or some discernible property associated with them. For instance, we could use colored rectangular boxes or upright bars as widgets. Each bar has a height and width. Ancient Greek mathematicians conceptualized numbers as measurable geometric properties, specifically the distance between two points. Building on this idea, we can treat each dimension of a bar as a number. However, for our animation, which includes a stop-and-move video transition along the x-axis, we will use the length of the bar to represent a different number and draw it upright, using its length for the vertical dimension, or y-axis.

We need to adjust the position of a bar whenever a corresponding number is manipulated programmatically through the sorting algorithm. Each small movement within the sorting process represents a single update to the collection of bars, occurring when one bar is exchanged with its neighboring bar and moved to its final position during a single iteration. This incremental movement is facilitated by a function called `after()`. In essence, the step-by-step transitions are captured by this procedure. The swapping action takes place at each step of every iteration, contributing to the overall animation.

Associating Numbers with Bars

A taller bar signifies a larger value in comparison to a shorter bar. The height of each bar is determined by referencing its y -coordinate against y_0 . Assuming the canvas is structured as a rectangle measuring 750 by 500 pixels, the maximum height achievable is 400 pixels. This means that the top-left and bottom-right corners of the canvas are positioned at (0,0) and (750,500), respectively. Let (x_t, y_t) denote the top-left corner and (x_b, y_b) the bottom-right corner of a bar. As the bar

is oriented vertically, its height can be expressed as $500 - y_t$. In essence, a taller bar is positioned nearer to the top of the canvas than a shorter one.

Redrawing Bars

Let us understand how the bars can be redrawn after a movement.

```
for i, value in enumerate(data):
    x0 = i * bar_width
    y0 = 400 - value
    x1 = x0 + bar_width - 2
    y1 = 400
    canvas.coords(bars[i], x0, y0, x1, y1)
```

The above loop goes through each number in the 'data' and updates the position of its corresponding bar.

In a Tkinter program, 'mainloop' handles screen updates, but during animations, the GUI needs to be refreshed programmatically after each step.

How Does It All Move?

In order to visualize the sorting process of the Bubble Sort algorithm, we use a function called 'bubble_sort_step()', which executes one step of the algorithm and pauses for a brief moment. This pause is facilitated by the 'after()' method, which schedules the next sorting operation after a short delay—typically around 100 milliseconds. This delay enhances the visual experience, creating a smooth animation effect as the algorithm sorts the data.

The sorting process involves two nested loops: the outer loop, represented by 'i', determines how often the algorithm will traverse the list, while the inner loop, indicated by 'j', is responsible for comparing adjacent numbers and swapping them if necessary. Once a comparison is finished, the 'after()' method is utilized to move to the next comparison pair ('j + 1'). After the inner loop completes, 'after()' is invoked again to initiate the next round of the outer loop ('i + 1').

As the Bubble Sort algorithm progresses and all sorting steps are completed, the visual representation shows the bars arranged in ascending order of their height on the canvas. To indicate that the sorting has been successfully completed, the bars change color to green.

The `'after()'` function acts much like a "pause" button during the sorting process, allowing Tkinter the necessary time to update the display after each step. By breaking down the sorting into smaller increments, the program maintains responsiveness, making the animation engaging and smooth.

An analogy for this process is that of a conveyor belt in a baggage claim area, where baggage handlers arrange luggage. Each time a handler swaps two pieces of luggage, the change is visible on the belt. The `'after()'` function serves as a pause between swaps, giving observers a chance to witness the sorting action unfold.